

进程间的通信方式

管道

- 匿名管道 pipe（亲缘通信）

```
ps aux | grep mysql//将|前命令的输入作为后一个命令的输入，实际是创建了2个子进程
```

创建一个匿名管道

```
pipe(文件描述符数组fds);//fds[2]:0 读 1写
```

在内核层使用一块共享的内存空间作为 环形管道缓冲区，缓存通信数据

原理：`fork` 创建子进程，**创建的子进程会复制父进程的文件描述**，那么父子进程可以各自使用一个文件描述符，向同一个管道中写入或读取数据

- 有名管道（fifo）

实现无亲缘关系进程间通信

创建管道文件

```
mkfifo name
```

```
mkfifo(char * name,0664)
```

原理：提前创建了一个类型为管道且有名字的设备文件，在进程里只要使用这个设备文件，就可以相互通信，因此不强调血缘关系；

总结

- 管道中的数据无格式
- 大小受限（环形队列底层实际就是一个数组去实现——连续的内存页）
- 单向通信（双向通信必须创建两个管道）且必须读取数据，写进程才会退出，否则就会阻塞，通信效率低
- 有内核态与用户态之间的数据拷贝开销

消息队列

解决了管道通信无格式、效率低的问题，只要写进程将数据成功写入了消息队列就可以成功退出；

原理：消息队列实际上就是保存在内核中的一个消息链表；链表上的每一个节点都是由通信双方提前确定的有格式消息体，进程读取消息体就会删除这个节点；

总结

- 有格式消息体
- 不适合传输大数据，因为内核对每个消息体（MSGMAX）和队列(MSGMNB)都有最大长度的限制
- 有内核态与用户态之间的数据拷贝开销

共享内存mmap

解决了内核态与用户态之间的数据拷贝开销的问题

```
#include<sys/mman.h>
ptr=mmap(NULL, size, PROT_READ | PROT_WRITE | PROT_EXEC | PROT_NONE, MAP_SHARED | MAP_PRIVATE, fd, offset);

mumap(void* ptr, in size); //释放内存映射
```

pram1，传null表示系统再进程中找到无人占用的内存当作映射内存

pram2，映射大小，一般是映射文件大小，不能为0

pram3, 映射内存权限指定

pram4，映射方式 分为私有映射和共享映射

pram 5, 映射文件的描述符

pram 6, 映射偏移量，不偏移为0

RETURN；成功返回映射内存地址，失败返回 MAP_FAILED关键字

原理：内存管理是使用虚拟内存技术，只要将进程AB的某一块虚拟内存映射到同一块物理内存上，两个进程就都能访问相同的一块物理内存，对这块内存的修改对方立刻能看见，不需要再拷贝；

总结

- 可以得出 由于OS采用的是虚拟内存技术对进程内存进行管理，因此即使进程的虚拟地址一样，只要访问不同的物理地址都没什么问题
- 解决内核态与用户态之间的数据拷贝开销
- 多个进程修改，后面的修改会覆盖前面的（出现脏页技术）

信号量（P V操作）

解决了mmap的多进程竞争共享资源造成的数据混乱问题

原理：信号量实质是一个资源计数器

- P操作会对信号量-1，**操作后**如果信号量<0,表示资源被占用，进程需要阻塞等待；>=0，资源可用，正常执行（先检测再使用）；
- V操作对信号量+1，**操作后**如果信号量<=0，说明至少有一个进程阻塞，唤醒进程；>0,不处理；

信号量初始化为1 表示互斥信号量，同时只能一个进程访问；（P V P V）

信号量初始化为0 表示同步信号量，读 P等待写 V；

信号

1. 前台进程：键盘组合

```
Ctrl+C //SIGINT, 终止当前进程  
Ctrl+Z //SIGTSTP, 停止当前进程
```

2. 后台进程：kill

```
kill -signo PID
```

3. 函数触发

```
kill(pid_t pid, int signo);
```

是异步通信

对信号的处理方式：

1. 信号默认

2. 信号捕捉

```
struct sigaction sa;

// 设置信号行为结构体
sa.sa_handler = signal_handler; // 指定自定义信号处理函数
sigemptyset(&sa.sa_mask);      // 清空信号屏蔽字
sa.sa_flags = 0;

sigaction(SIGINT, &sa, NULL);
```

3. 信号忽略 使用 `signal` 或 `sigaction` 将信号处理函数设置为 `SIG_IGN`

```
// 忽略SIGINT信号
signal(SIGINT, SIG_IGN);
```

信号失效的三种方式

- 忽略：是永久的，用于完全忽略某些信号，这些信号不会对进程产生任何影响，也不会被记录在未决信号集中。

实现：使用 `signal` 或 `sigaction` 将信号处理函数设置为 `SIG_IGN`

```
// 忽略SIGINT信号
signal(SIGINT, SIG_IGN);
```

- 屏蔽：是暂时的，用于延迟信号处理，避免信号处理函数中断当前操作。被屏蔽的信号会保留在未决信号集中，直到屏蔽被解除。

实现：使用 `sigprocmask` 设置信号屏蔽字

```
// 屏蔽SIGINT信号
sigaddset(&set, SIGINT);
sigprocmask(SIG_BLOCK, &set, NULL);
```

- 捕捉函数：捕获信号并执行自定义忽略处理逻辑

```
struct sigaction sa;
// 设置SIGINT信号的处理方式为忽略
sa.sa_handler = SIG_IGN; // 忽略
sigemptyset(&sa.sa_mask); // 清空信号屏蔽字
sa.sa_flags = 0;          // 默认行为
```

```
sigaction(SIGINT, &sa, NULL); //不保存之前的
```

*** 所有信号都失效，系统是否安全，如何保证系统可以杀死进程**

普通信号

系统发送某些信号例如 SIGSEGV SIGFPE 信号，权限较高，无法屏蔽

高级信号

SIGKILL (9) 只要发出必然递达进程，杀死进程，无法屏蔽、忽略、捕捉

SIGSTOP(19) 只要发出必然挂起进程，无法失效处理

socket

跨网，主要用于实现不同主机上的进程通信（TCP UDP）（也可以实现相同主机上不同进程间通信）

小结

线程之间不必关注通信的方式，因为只要有共享的内存就可以实现通信；线程是共享进程的资源，天然具有可以实现通信的基础，所以应该考虑的是竞争共享资源的问题；